

# **SYSTEM DEVELOPMENT**

## **SYSTEM DEVELOPMENT**

A software implementation method is integrating software-based services systematically. This is the phase in the software life cycle where the actual software is implemented. The result of this phase consists of source code, together with documentation to make the code more readable. Implementation is the stage of a project where the theoretical design is turned into a working system. If the implementation is not carefully planned and controlled, it may cause confusion.

The most crucial stage is achieving a successful system and giving users confidence that the system will work according to their requirements. It involves careful planning, investigation of the current system, and its constraints. The more complex the system, the more effort is needed to implement it. In other words, implementation is the process of bringing a developed system into operational use and turning it over to the user. Implementation activities extend from planning through converting the old system to the new one.

The system development of the LearnX AI-Driven Personalized E-Learning Platform was executed through an Agile and iterative process focused on rapid deployment, personalized learning experiences, and highly intuitive educational user interfaces. The initial planning phase involved comprehensive requirement gathering to define core learning needs like real-time instructor-learner matching, automated attendance tracking, accurate AI-driven diagnostic assessment, and personalized remedial content recommendation. The system design prioritized a scalable, web-based architecture utilizing a cloud-hosted database for user and progress management, a Node.js backend API for business logic orchestration, a third-party video streaming API for live sessions, and an AI microservice for intelligent diagnostics and reporting. Agile methodologies were employed to support continuous development, allowing the team to incrementally build, test, and refine modules such as the personalized learner dashboard, the intelligent session scheduling system, and the AI diagnostic reporting engine. Rigorous unit and integration tests were conducted to guarantee seamless data flow and correct diagnostic evaluations. The resulting system is a robust, intelligent, Web-based learning solution that delivers high efficiency, accurate personalized assessments, and seamless instructor-learner interaction.

### **1. ADMINISTRATOR DASHBOARD**

This module provides the central interface for the Platform Owner or Administrator, offering high-level operational and platform configuration oversight. It aggregates key data like total active learners count, active instructors count, ongoing live sessions, platform-wide attendance summaries, and AI diagnostic report generation statistics. It also provides

tools for instructor verification and approval, global subscription plan configuration, and system announcement broadcasting.

## **2. LEARNER AND INSTRUCTOR MANAGEMENT**

This module is responsible for the complete lifecycle of user accounts and role management. It includes tools for learner registration, instructor onboarding and verification, role-based access assignment, profile updates, and account status monitoring. It is the primary module for generating user capacity indicators, tracking active versus inactive users, and applying role-specific permission filters.

## **3. INTELLIGENT INSTRUCTOR-LEARNER MATCHING MODULE**

This module manages the intelligent matching system, including learner preferences, instructor expertise profiles, and matching histories. Crucially, it tracks the compatibility relationships between learners and instructors based on subject alignment, availability constraints, learning style compatibility, and past performance data, allowing users to view and receive automated instructor recommendations.

## **4. LIVE SESSION MANAGEMENT MODULE**

The core user interface for scheduling, hosting, and joining live interactive sessions. It features a rapid scheduling pipeline with integrated calendar view, real-time availability checking, and automatic attendance tracking functionality. It handles session creation, instructor assignment, learner enrollment, session recording capture, automatic join time logging, duration tracking, and final session completion reporting.

## **5. AI-DIAGNOSTIC ASSESSMENT MODULE**

This intelligence-focused module handles automated learner performance analysis. It calculates and generates detailed diagnostic reports after each completed session, identifying specific knowledge gaps, weak areas, and mastery levels. It automatically pushes personalized remedial content recommendations, including suggested video modules, practice quizzes, and one-on-one revision session requests, ensuring seamless continuous improvement.

## **6. PROGRESS AND ATTENDANCE TRACKING MODULE**

This background execution process analyzes learner session metrics against completion benchmarks and automatically pushes attendance records and progress updates to the dashboard interface. It tracks join times, participation duration, engagement metrics, session completion rates, assessment scores, and overall mastery trends, minimizing manual tracking efforts and ensuring accurate learning journey documentation.

## **7. CONTENT REPOSITORY & LEARNING HISTORY AUDIT**

Focusing on absolute accountability, this module compiles comprehensive tracking records of all content accessed, sessions completed, diagnostic reports generated, and remedial materials consumed. It relies on immutable database properties to deliver a verifiable learning audit trail essential for learner verification and ensuring total system integrity

## **8. SOFTWARE DEVELOPMENT**

- Frontend: Implemented using HTML5, CSS3, Bootstrap, React.js or Vanilla JavaScript to ensure highly responsive web navigation panels and a seamless user experience across devices.
- Architecture: The application operates as a single-page web application (SPA) with integrated video streaming APIs and real-time communication protocols for fast, frictionless learner-instructor interaction.
- Backend Integration: Server-side logic is achieved via a Node.js/Express API layer handling user authentication, session scheduling, attendance recording, AI diagnostic report generation, and content repository management.

## **9. DATABASE DESIGN**

- Primary Database: A cloud-hosted relational database (such as PostgreSQL or MySQL) is utilized as the core persistent storage, providing structured tables for users, roles, sessions, attendance records, diagnostic reports, content metadata, and progress tracking, accessed via optimized API queries.
- File Storage Pipeline: The frontend and Node.js backend use API calls to route video recordings and pre-recorded content directly into cloud storage services (such as AWS S3 or similar), linking structural records back to the primary database using unique file identifiers and access URLs.

## **10. USER INTERFACE**

- Intuitive and responsive UI design modeled after conventional cloud drives for fast folder and file interaction in the storage grid.
- Dashboard UI designed to clearly display storage tier KPIs, file category cards, and MetaMask connectivity badges.

## **11. TESTING AND EVALUATION**

- Unit Testing: Executed for critical smart contract functions like gas limit optimizations, free tier constraint boundaries, and purchase allocation logic.
- Integration Testing: Conducted to ensure seamless data handoffs between the local Node.js proxy server, Pinata IPFS endpoints, and the MetaMask transaction window injection.

## **SYSTEM TESTING AND IMPLEMENTATION**

## **SYSTEM TESTING**

In general, testing is finding out how well software or a product works. Testing is a set of activity that can be planned and conducted systematically. Testing begins at the module level and works towards the integration of entire computer-based systems. Nothing is complete without testing, as it is vital to the success of the system.

Software testing is a process of executing a program or application with the intent of finding the software bugs. Testing is for evaluating a system or its components to find whether it satisfies the specified requirements or not. Testing cannot show the absence of defects; it can also be stated as the process of validating and verifying a software program, system, or product.

## **TESTING TECHNIQUES**

- Unit Testing
- Integration Testing
- Validation Testing
- System Testing
- Output Testing
- User Acceptance Testing

## **UNIT TESTING**

It is the first level of testing. Each module is tested individually, and focus is given to finding errors limited to each individual module and correcting them. The different modules of the system are tested individually and corrected for all errors. Each module is focused on working satisfactorily with regard to the expected output from the module.

Here, we verify the proper functioning of individual components like the user login authentication hook, live session scheduling logic, attendance tracking calculation, and AI diagnostic report generation engine. For Example: Testing the instructor-learner matching algorithm to ensure that trying to match a learner with an incompatible instructor based on subject misalignment or availability conflict accurately prevents the match and provides appropriate alternative recommendations.

## **INTEGRATION TESTING**

Integration testing is a systematic test for constructing the program structure while at the same time conducting tests to uncover errors. Instead of testing the system as a whole, it focuses on how the modules that make up the system interact. It tests for the errors resulting

from the integration of modules. One specific target of integration testing is the interface between modules. The objective is to take unit-tested modules and build a program structure dictated by design.

Here, we check whether modules like the Live Session Management, Intelligent Instructor-Learner Matching, and the AI Diagnostic Assessment engine work together as expected. For Example: Ensuring that a successful live session completion accurately triggers the AI diagnostic engine to analyze learner performance, generates a detailed weakness report, automatically updates the learner dashboard with attendance records, and pushes personalized remedial content recommendations to the learner's profile.

### **VALIDATION TESTING**

Validation testing is performed to ensure the system meets the functional requirements specified by the user. For instance, checking the Contract Administrator dashboard to ensure role-based settings modifications are correctly granted or denied based on the connected public key. Ensure all critical features, such as Decentralized Marketplace operations and user file sorting filters, operate correctly.

For Example: Checking that executing a file purchase transaction within the Marketplace module accurately calculates token amounts in Wei, transfers owner rights on the blockchain state ledger, updates access variables, and instantly shifts the bought asset into the recipient's "My Files" inventory.

### **SYSTEM TESTING**

System testing was performed to verify that all system elements have been properly integrated and perform their allocated function within the complete system environment. Security testing was done to check the security mechanisms built into the system, which will protect it from improper penetration (e.g., ensuring unwhitelisted Ethereum addresses cannot decrypt or view private IPFS hashes). Performance testing was done to test the runtime performance of the software. For user acceptance testing, the system was given to the end-user to use. Confirm that the entire app operates seamlessly, including both Administrator and Standard User functionalities.

For Example: Ensuring that the web application handles multiple concurrent standard user uploads and heavy IPFS payload lookups across diverse browsers using MetaMask extensions without stalling the interface or encountering Web3 RPC data concurrency errors.

### **OUTPUT TESTING**

After performing the validation testing, the next step is the output testing of the enhanced system. No system could be useful if it does not produce the required output in the required format. The outputs generated or displayed by the system are tested by asking the users about the format required by them. Check that outputs like the file history log table, storage capacity progress cards, and marketplace directory displays present accurate business and ledger data based on user inputs.

For Example: Ensuring that the generated on-chain history record displays the correct transaction hash string, the precise byte size, the corresponding asset file name, and an accurate Unix block timestamp of the event.

## **USER ACCEPTANCE TESTING**

User acceptance testing comprises a completed and successful end-to-end system test with review of the results by one or more users with specific knowledge. Users may apply a variety of validation techniques. For example: generating a transaction report from the new system and comparing the results with the same ledger state from the test network explorer, data inspection, and others. Prior to this testing, the system was delivered to the clients along with the steps for implementation.

In user acceptance testing, the system was run in the client environment by system users. The users were allowed to test the system and raise any issues for a specific period after the system is assumed to be complete and error-free. Here, we collect feedback from Administrators and Users about the application's functionality and usability.

For Example: Allowing a test user to connect their MetaMask wallet, upload an asset payload using the user dashboard interface, designate a second test wallet to a file access whitelist mapping, and have that second user independently verify successful download resolution of the pinned document to ensure all decentralized actions work seamlessly for storage operations.

## **SYSTEM IMPLEMENTATION**

A software implementation method is integrating software-based services systematically. This is the phase in the software life cycle where the actual software is implemented. The result of this phase consists of source code, together with documentation to make the code more readable. Implementation is the stage of a project where the theoretical design is turned into a working system. If the implementation is not carefully planned and controlled, it may cause confusion.

Implementation is done only after testing is complete. The most crucial stage is achieving a new successful system and giving confidence about the new system to the user that it will work according to the requirements. It involves careful planning, investigation of the current system, and its constraints on implementation. The more complex the system is, the more effort is needed to implement the system. In other words, implementation is the



process of bringing a developed system into operational use and turning it over to the user. Implementation activities extend from planning through the old system to the new one.

The implementation involves the following formalities:

- **Careful planning:** Formulating structured milestones for compiling the Solidity source contracts via Hardhat, environment variable synchronization for Pinata API endpoints, and main deployment procedures.
- **Investigation of the systems and constraints:** Evaluating browser Web3 environments and handling gas fee parameters on the Sepolia network test layers to minimize user transaction failures.
- **Design the methods to achieve the changes:** Structuring proxy backend server gateways and Ethers.js integration listeners to securely transition user asset workflows from legacy local structures to a decentralized paradigm.
- **Training the staffs in the changed phase:** Providing instructional guides to users regarding MetaMask setup instructions, secure private key preservation rules, and transaction confirmation flows.
- **Evaluation of the changes over method:** Reviewing live storage performance metrics, gas optimizations, and repository consistency records against initial user privacy and sovereign expectations.

# **SYSTEM MAINTENANCE**

## **SYSTEM MAINTENANCE**

A process of modifying a software system or component after deployment to correct faults, improve performance, and adapt to changing requirements is known as software maintenance. Software maintenance is not limited to fixing defects; it also includes enhancing existing features, optimizing system performance, removing obsolete functionalities, and ensuring compatibility with evolving technologies. Since changes in software environments are inevitable, maintenance activities involve evaluating, controlling, and implementing modifications throughout the system's operational life. The primary objective of maintenance is to preserve and enhance the value of the software over time by improving usability, efficiency, security, and functionality. While software development may take only a few years, maintenance can continue for a much longer period to ensure the system remains reliable and relevant.

For the LEARNX AI-Driven Personalized E-Learning Platform , corrective maintenance is performed to resolve faults and errors encountered during daily operation, such as live video streaming interruptions, AI diagnostic engine processing failures, real-time chat synchronization issues, attendance tracking inaccuracies, and database query errors that affect learner progress retrieval and dashboard rendering.

Adaptive maintenance is carried out to accommodate changes in external technologies and services, including updates to third-party video conferencing APIs (such as Zoom, Daily, or LiveKit), new versions of WebRTC protocols for real-time communication, changes in cloud storage service providers for recorded session archiving, modifications to AI model deployment frameworks, and evolving browser security policies affecting WebRTC and media capture functionalities.

Perfective maintenance focuses on improving system performance and user experience by optimizing the instructor-learner matching algorithm for faster and more accurate recommendations, reducing AI diagnostic report generation latency, enhancing dashboard responsiveness across devices, refining the user interface for better accessibility and navigation, improving video streaming quality under varying network conditions, and implementing smarter remedial content recommendation logic based on longitudinal learner performance data.

Preventive maintenance is undertaken to reduce the possibility of future failures by conducting regular security audits of user data storage, identifying potential vulnerabilities in authentication mechanisms, monitoring server load and auto-scaling thresholds, implementing automated health checks for all critical services (video streaming, AI engine, database, notification system), performing database backups and integrity verification, stress-testing the platform under high concurrency scenarios, and regularly updating dependencies to patch known security vulnerabilities.

# CONCLUSION

## **CONCLUSION**

In conclusion, LEARNX addresses the fundamental shortcomings of traditional online learning platforms by offering an integrated, intelligent, and learner-centric ecosystem. By combining structured content delivery, real-time live interaction, intelligent instructor matching, and AI-driven diagnostic assessments, the platform effectively eliminates the inefficiencies of manual coordination and surface-level evaluation. It empowers learners across all categories to identify and bridge their knowledge gaps systematically while providing a consolidated view of their attendance and progress. The automation of critical processes such as weakness analysis and remediation recommendations ensures that learning becomes more personalized, efficient, and outcome-driven. Ultimately, LEARNX transforms the educational journey from a fragmented, time-consuming experience into a seamless, adaptive, and mastery-focused process, fulfilling the growing demand for intelligent solutions in the rapidly evolving landscape of digital education.

## **SCOPE FOR FUTURE ENHANCEMENT**

Future enhancements will significantly expand LEARNX's capabilities beyond core

personalized learning, live interaction, and AI-driven assessment. A key focus is introducing **\*\*Blockchain-Based Credentialing and Certification\*\***, enabling learners to receive tamper-proof, verifiable digital certificates and micro-credentials upon successful completion of courses or mastery of specific subjects. These credentials can be shared directly with employers or academic institutions, eliminating manual verification processes.

The system can be scaled to support Multi-Institution and Enterprise Learning Repositories, allowing schools, universities, and corporate training departments to create their own segregated learning environments under a single platform instance. This includes centralized administration, cross-institution analytics, and shared instructor pools while maintaining data isolation and privacy compliance.

Further upgrades include seamless integration with **Advanced Adaptive Learning Algorithms\*\*** powered by reinforcement learning, which will dynamically adjust remedial content difficulty and pacing in real-time based on continuous learner performance monitoring. This moves beyond static diagnostic reports to truly personalized, self-optimizing learning pathways.

The addition of Gamification and Peer Learning Engines will boost learner engagement and retention through experience points (XP), achievement badges, leaderboards, and structured peer-to-peer study groups with collaborative problem-solving sessions. Learners will be able to form virtual study circles, share notes, and conduct peer reviews under instructor moderation.

Integration of Natural Language Processing (NLP) for Intelligent Chatbots will provide learners with 24/7 academic support, answering subject-related queries, explaining concepts, and guiding them through remedial content without requiring live instructor intervention. This significantly reduces response times for common questions and enhances the self-service learning experience.

Finally, integration with Learning Tools Interoperability (LTI) Standards will allow LEARNX to connect with existing Learning Management Systems (LMS) such as Moodle, Canvas, and Blackboard, enabling institutions to adopt LEARNX's AI-driven diagnostic and live session capabilities without abandoning their current infrastructure. This ensures the system evolves into a comprehensive, interoperable, and future-proof intelligent learning ecosystem.

# BIBLIOGRAPHY

## BIBLIOGRAPHY

## **WEBSITES**

- <http://chatgpt.openai.com>
- <https://docs.ipfs.tech/>
- <https://docs.soliditylang.org/>
- <https://docs.pinata.cloud/>
- <https://docs.ethers.org/v6/>

## **BOOKS**

- Online education : foundations ,planning and pedagogy
- Digital learning design:Designing Effective Online Learning Design
- Small Teaching online:Applying Learning Science in online Classes



# APPENDIX

## SOURCE CODES

### SAMPLE

```
6 class ApiService {
12     Future<void> register(
23         'role': role,
24         'categories': categories,
25     )),
26     headers: {'Content-Type': 'application/json'},
27 );
28 if (res.statusCode != 200) throw Exception(res.body);
29 final body = jsonDecode(res.body) as Map<String, dynamic>;
30 currentEmail = email;
31 currentRole = role;
32 currentCategories = List<String>.from(body['user']['categories'] ?? []);
33 }
34
35 Future<void> login(String email, String password) async {
36     final res = await http.post(
37         Uri.parse('$baseUrl/auth/login'),
38         body: jsonEncode({'email': email, 'password': password}),
39         headers: {'Content-Type': 'application/json'},
40     );
41     if (res.statusCode != 200) throw Exception(res.body);
42     final body = jsonDecode(res.body) as Map<String, dynamic>;
43     currentEmail = email;
44     currentRole = body['user']['role'] as String?;
45     currentCategories = List<String>.from(body['user']['categories'] ?? []);
46 }
47
48 static void logout() {
49     currentEmail = null;
50     currentRole = null;
51     currentCategories = [];
52 }
53
54 Future<List> fetchVideos({List<String>? types}) async {
55     final params = <String, String>{};
56     if (types != null && types.isNotEmpty) {
```

```

body: SingleChildScrollView(
  padding: const EdgeInsets.all(16.0),
  child: Form(
    key: _formKey,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        TextFormField(
          decoration: const InputDecoration(
            labelText: 'Email',
            border: OutlineInputBorder(),
          ), // InputDecoration
          validator: (v) =>
            (v == null || v.isEmpty) ? 'Enter email' : null,
          onSave: (v) => email = v ?? '',
        ), // TextFormField
        const SizedBox(height: 16),
        TextFormField(
          decoration: const InputDecoration(
            labelText: 'Password',
            border: OutlineInputBorder(),
          ), // InputDecoration
          obscureText: true,
          controller: _passwordController,
          validator: (v) =>
            (v == null || v.isEmpty) ? 'Enter password' : null,
        ), // TextFormField
      ],
    ),
  ),
)

```

```

}

class _TeacherHomePageState extends State<TeacherHomePage>
  with SingleTickerProviderStateMixin {
  final _formKey = GlobalKey<FormState>();
  late TabController _tabController;
  String title = '';
  String url = '';
  File? selectedFile;
  List myVideos = [];

  @override
  void initState() {
    super.initState();
    _tabController = TabController(length: 2, vsync: this);
    _loadMyVideos();
  }

  @override
  void dispose() {
    _tabController.dispose();
    super.dispose();
  }

  Future<void> _loadMyVideos() async {
    final api = ApiService();
  }
}

```

SCREENSHOTS

